```
import logging
import json
import os
from datetime import datetime
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler, MessageHandler, CallbackQueryHandler,
    ContextTypes, filters, ConversationHandler
)

# === НАСТРОЙКИ ===
BOT_TOKEN = "8779626555:AAEiIrpRSj5UFZY9xlgDywKfn4t420zi6ew"
ADMIN_ID = 7900167281
CATALOG_URL = "https://t.me/projectkatalog"
ORDERS_FILE = "orders.json"

# === КАЛЬКУЛЯТОР ЦЕН ===
YUAN_RATE = 12
MY_MARKUP = 750

# === РЕКВИЗИТЫ ===
PAYMENT_DETAILS = (
    "💳 *Реквизиты для оплаты:*\n\n"
    "Банк: ВТБ\n"
    "Номер: +7 978 556-28-24\n"
    "Получатель: Артур М.\n\n"
    "⚠️ После оплаты отправь скриншот менеджеру @projectmanag3r"
)

# === СОСТОЯНИЯ ===
(
    MAIN_MENU,
    ORDER_PHOTO,
    ORDER_PHOTO_WAIT_PRICE,
    ORDER_PHOTO_SIZE,
    ORDER_PHOTO_SIZE_WAIT,
    ORDER_PHOTO_COLOR,
    ORDER_PHOTO_QUANTITY,
    ORDER_PHOTO_CONTACTS,
    ORDER_PHOTO_CONFIRM,
    DELIVERY_CALC,
) = range(10)

logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger(__name__)

# ============================================================
# БАЗА ЗАКАЗОВ
# ============================================================

def load_orders():
    if os.path.exists(ORDERS_FILE):
        with open(ORDERS_FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    return {}

def save_orders(orders):
    with open(ORDERS_FILE, "w", encoding="utf-8") as f:
        json.dump(orders, f, ensure_ascii=False, indent=2)

def add_order(order_id, data):
    orders = load_orders()
    orders[order_id] = data
    save_orders(orders)

def get_order(order_id):
    return load_orders().get(order_id)

def update_order_status(order_id, status):
    orders = load_orders()
    if order_id in orders:
        orders[order_id]["status"] = status
        orders[order_id]["updated"] = datetime.now().strftime("%d.%m.%Y %H:%M")
        save_orders(orders)
        return True
    return False

def get_user_orders(user_id):
    orders = load_orders()
    return {oid: o for oid, o in orders.items() if o.get("user_id") == user_id}

def generate_order_id():
    return f"ORD-{len(load_orders()) + 1:04d}"

STATUS_LABELS = {
    "new":       "🆕 Новый",
    "confirmed": "✅ Подтверждён",
    "paid":      "💰 Оплачен",
    "in_china":  "🇨🇳 На складе в Китае",
    "shipping":  "✈️ В пути",
    "arrived":   "📦 Прибыл",
```

```
    "delivered": "🎉 Доставлен",
    "cancelled": "❌ Отменён",
}

def calc_price(yuan):
    return int(yuan * YUAN_RATE + MY_MARKUP)

# ============================================================
# ГЛАВНОЕ МЕНЮ
# ============================================================

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data.clear()
    keyboard = [
        [InlineKeyboardButton("🛍️ Каталог наличия", callback_data="catalog")],
        [InlineKeyboardButton("📸 Заказать из Китая", callback_data="order_photo"
        [InlineKeyboardButton("🚚 Рассчитать доставку", callback_data="delivery")
        [InlineKeyboardButton("📋 Мои заказы", callback_data="my_orders")],
        [InlineKeyboardButton("❓ Частые вопросы", callback_data="faq")],
        [InlineKeyboardButton("📞 Менеджер", callback_data="manager")],
    ]
    text = "👋 Привет! Я бот телеграмм канала PROJECT\n\nВыбери, что тебя интерес
    if update.message:
        await update.message.reply_text(text, reply_markup=InlineKeyboardMarkup(k
    else:
        await update.callback_query.edit_message_text(text, reply_markup=InlineKe
    return MAIN_MENU

# ============================================================
# КАТАЛОГ
# ============================================================

async def catalog(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    keyboard = [[InlineKeyboardButton("⬅️ Назад", callback_data="back")]]
    await query.edit_message_text(
        f"📦 Наш каталог с наличием:\n\n{CATALOG_URL}\n\n"
        "Все актуальные позиции там. Если нужного размера нет — закажем из Китая
        reply_markup=InlineKeyboardMarkup(keyboard)
    )
    return MAIN_MENU

# ============================================================
# ЗАКАЗ ПО ФОТО
# Шаги: фото → ждём цену → размер (или запрос сетки) → цвет → кол-во → контакт →
# ============================================================
```

```
async def order_photo_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    await query.edit_message_text(
        "📸 *Заказ из Китая — Шаг 1/7*\n\n"
        "Отправь фото вещи, которую хочешь заказать.\n\n"
        "Мы найдём её на Poizon, Pinduoduo, Taobao, 1688, Gofish, 95 и пришлём це
        parse_mode="Markdown"
    )
    return ORDER_PHOTO

async def order_photo_receive(update: Update, context: ContextTypes.DEFAULT_TYPE)
    user = update.message.from_user
    now = datetime.now().strftime("%d.%m.%Y %H:%M")

    if update.message.photo:
        context.user_data['photo_file_id'] = update.message.photo[-1].file_id
        context.user_data['photo_type'] = 'photo'
    elif update.message.document:
        context.user_data['photo_file_id'] = update.message.document.file_id
        context.user_data['photo_type'] = 'document'
    else:
        await update.message.reply_text("❌ Пожалуйста, отправь именно фото.")
        return ORDER_PHOTO

    context.user_data['user_id'] = user.id
    context.user_data['username'] = user.username or "нет"
    context.user_data['full_name'] = user.full_name

    caption = (
        f"📸 *НОВЫЙ ЗАПРОС — ПОИСК ТОВАРА* — {now}\n\n"
        f"👤 {user.full_name} | @{user.username or 'нет'} | ID: {user.id}\n\n"
        "Найди товар на Poizon / Pinduoduo / Taobao / 1688 / Gofish / 95\n\n"
        "Узнай цену в юанях и отправь:\n"
        f"`/price {user.id} <цена в юанях>`\n\n"
        f"Пример: `/price {user.id} 350`"
    )

    if context.user_data['photo_type'] == 'photo':
        await context.bot.send_photo(
            chat_id=ADMIN_ID,
            photo=context.user_data['photo_file_id'],
            caption=caption,
            parse_mode="Markdown"
        )
    else:
```

```
        await context.bot.send_document(
            chat_id=ADMIN_ID,
            document=context.user_data['photo_file_id'],
            caption=caption,
            parse_mode="Markdown"
        )

    await update.message.reply_text(
        "✅ Фото получено!\n\n"
        "⏳ Ищем твой товар на Poizon, Pinduoduo, Taobao, 1688, Gofish, 95...\n\n
        "Как только найдём — пришлём цену. Обычно это занимает несколько минут."
    )
    return ORDER_PHOTO_WAIT_PRICE

async def order_photo_waiting(update: Update, context: ContextTypes.DEFAULT_TYPE)
    await update.message.reply_text("⏳ Ожидай — менеджер уже ищет товар. Скоро п
    return ORDER_PHOTO_WAIT_PRICE

# ============================================================
# ADMIN: /price <user_id> <цена в юанях>
# ============================================================

async def admin_send_price(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if update.effective_user.id != ADMIN_ID:
        return

    args = context.args
    if len(args) < 2:
        await update.message.reply_text(
            "Использование: /price <user_id> <цена в юанях>\n\nПример: /price 123
        )
        return

    try:
        client_id = int(args[0])
        yuan_price = float(args[1])
    except ValueError:
        await update.message.reply_text("❌ Неверный формат. Пример: /price 12345
        return

    rub_price = calc_price(yuan_price)

    if 'pending_prices' not in context.bot_data:
        context.bot_data['pending_prices'] = {}
    context.bot_data['pending_prices'][client_id] = {
        'yuan': yuan_price,
        'rub': rub_price
```

```
    }

    keyboard = [
        [InlineKeyboardButton("✅ Продолжить заказ", callback_data="photo_price_c
        [InlineKeyboardButton("❌ Отмена", callback_data="back")],
    ]

    await context.bot.send_message(
        chat_id=client_id,
        text=(
            f"✅ *Нашли твой товар!*\n\n"
            f"💰 Стоимость товара: *{rub_price} ₽*\n"
            f"_({yuan_price:.0f} ¥ × {YUAN_RATE} + {MY_MARKUP} ₽ наценка)_\n\n"
            "Хочешь оформить заказ?"
        ),
        reply_markup=InlineKeyboardMarkup(keyboard),
        parse_mode="Markdown"
    )

    await update.message.reply_text(f"✅ Цена {rub_price} ₽ отправлена клиенту {c

async def photo_price_ok(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    user_id = query.from_user.id

    pending = context.bot_data.get('pending_prices', {}).get(user_id)
    if pending:
        context.user_data['yuan_price'] = pending['yuan']
        context.user_data['rub_price'] = pending['rub']

    keyboard = [
        [InlineKeyboardButton("❓ Не знаю размер", callback_data="size_unknown")]
    ]
    await query.edit_message_text(
        "Шаг 2/7 — Размер\n\n"
        "Укажи нужный размер _(S, M, L, XL, 42, EU 44...)_\n\n"
        "Не знаешь размер? Нажми кнопку ниже — пришлём размерную сетку.",
        reply_markup=InlineKeyboardMarkup(keyboard),
        parse_mode="Markdown"
    )
    return ORDER_PHOTO_SIZE

# ============================================================
# РАЗМЕР — клиент не знает размер → запрос сетки у менеджера
# ============================================================
```

```
async def size_unknown(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    user = query.from_user
    now = datetime.now().strftime("%d.%m.%Y %H:%M")

    # Уведомляем менеджера
    await context.bot.send_message(
        chat_id=ADMIN_ID,
        text=(
            f"📏 *ЗАПРОС РАЗМЕРНОЙ СЕТКИ* — {now}\n\n"
            f"👤 {user.full_name} | @{user.username or 'нет'} | ID: {user.id}\n\n
            "Клиент не знает размер. Отправь размерную сетку командой:\n\n"
            f"`/sizechart {user.id}`\n\n"
            "После этой команды бот запросит у тебя фото/файл сетки."
        ),
        parse_mode="Markdown"
    )

    await query.edit_message_text(
        "📏 Запросили размерную сетку у менеджера.\n\n"
        "⏳ Пришлём её тебе в течение нескольких минут!"
    )
    return ORDER_PHOTO_SIZE_WAIT

async def order_photo_size_waiting(update: Update, context: ContextTypes.DEFAULT_
    await update.message.reply_text("⏳ Ожидай — менеджер уже готовит размерную с
    return ORDER_PHOTO_SIZE_WAIT

# ============================================================
# ADMIN: /sizechart <user_id>
# Менеджер отправляет размерную сетку клиенту
# ============================================================

async def admin_sizechart_cmd(update: Update, context: ContextTypes.DEFAULT_TYPE)
    if update.effective_user.id != ADMIN_ID:
        return

    args = context.args
    if len(args) < 1:
        await update.message.reply_text(
            "Использование: /sizechart <user_id>\n\nПример: /sizechart 123456789"
        )
        return

    try:
        client_id = int(args[0])
```

```
    except ValueError:
        await update.message.reply_text("❌ Неверный user_id")
        return

    if 'pending_sizechart' not in context.bot_data:
        context.bot_data['pending_sizechart'] = {}
    context.bot_data['pending_sizechart'][ADMIN_ID] = client_id

    await update.message.reply_text(
        f"📏 Теперь отправь фото или файл размерной сетки для клиента {client_id}
        "Следующее фото/документ будет автоматически переслано клиенту."
    )

async def admin_sizechart_send(update: Update, context: ContextTypes.DEFAULT_TYPE
    """Менеджер отправляет фото/документ сетки — бот пересылает клиенту."""
    if update.effective_user.id != ADMIN_ID:
        return

    pending = context.bot_data.get('pending_sizechart', {})
    client_id = pending.get(ADMIN_ID)

    if not client_id:
        return  # нет активного запроса, игнорируем

    keyboard = [[InlineKeyboardButton("✅ Знаю размер, продолжить", callback_data

    if update.message.photo:
        await context.bot.send_photo(
            chat_id=client_id,
            photo=update.message.photo[-1].file_id,
            caption="📏 *Размерная сетка*\n\nВыбери свой размер и нажми кнопку ни
            reply_markup=InlineKeyboardMarkup(keyboard),
            parse_mode="Markdown"
        )
    elif update.message.document:
        await context.bot.send_document(
            chat_id=client_id,
            document=update.message.document.file_id,
            caption="📏 *Размерная сетка*\n\nВыбери свой размер и нажми кнопку ни
            reply_markup=InlineKeyboardMarkup(keyboard),
            parse_mode="Markdown"
        )

    # Очищаем ожидание
    context.bot_data['pending_sizechart'].pop(ADMIN_ID, None)
    await update.message.reply_text(f"✅ Размерная сетка отправлена клиенту {clie
```

```
async def size_known(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Клиент нажал 'Знаю размер' после просмотра сетки."""
    query = update.callback_query
    await query.answer()
    await query.edit_message_text(
        "Шаг 2/7 — Размер\n\nНапиши свой размер _(S, M, L, XL, 42, EU 44...)_",
        parse_mode="Markdown"
    )
    return ORDER_PHOTO_SIZE

async def order_photo_size(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data['size'] = update.message.text
    await update.message.reply_text(
        "Шаг 3/7 — Цвет\n\n_(чёрный, белый, синий...)_",
        parse_mode="Markdown"
    )
    return ORDER_PHOTO_COLOR

async def order_photo_color(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data['color'] = update.message.text
    await update.message.reply_text("Шаг 4/7 — Количество")
    return ORDER_PHOTO_QUANTITY

async def order_photo_quantity(update: Update, context: ContextTypes.DEFAULT_TYPE
    context.user_data['quantity'] = update.message.text
    await update.message.reply_text("Шаг 5/7 — Контакт\n\nUsername или номер теле
    return ORDER_PHOTO_CONTACTS

async def order_photo_contacts(update: Update, context: ContextTypes.DEFAULT_TYPE
    context.user_data['contacts'] = update.message.text
    d = context.user_data
    summary = (
        f"📋 *Ваш заказ — Шаг 6/7:*\n\n"
        f"💰 Стоимость товара: *{d.get('rub_price')} ₽*\n"
        f"📏 Размер: {d.get('size')}\n"
        f"🎨 Цвет: {d.get('color')}\n"
        f"🔢 Количество: {d.get('quantity')}\n"
        f"📞 Контакт: {d.get('contacts')}\n\n"
        "Всё верно?"
    )
    keyboard = [
        [InlineKeyboardButton("✅ Подтвердить", callback_data="photo_confirm")],
        [InlineKeyboardButton("❌ Отмена", callback_data="back")],
    ]
    await update.message.reply_text(summary, reply_markup=InlineKeyboardMarkup(ke
    return ORDER_PHOTO_CONFIRM
```

```
async def order_photo_confirm(update: Update, context: ContextTypes.DEFAULT_TYPE)
    query = update.callback_query
    await query.answer()
    user = query.from_user
    d = context.user_data
    now = datetime.now().strftime("%d.%m.%Y %H:%M")
    order_id = generate_order_id()

    add_order(order_id, {
        "user_id": user.id,
        "username": user.username or "нет",
        "full_name": user.full_name,
        "yuan_price": d.get('yuan_price'),
        "rub_price": d.get('rub_price'),
        "size": d.get('size'),
        "color": d.get('color'),
        "quantity": d.get('quantity'),
        "contacts": d.get('contacts'),
        "status": "confirmed",
        "date": now,
        "updated": now,
    })

    # Уведомление менеджеру
    await context.bot.send_message(
        chat_id=ADMIN_ID,
        text=(
            f"🛒 *ЗАКАЗ ПОДТВЕРЖДЁН {order_id}* — {now}\n\n"
            f"👤 {user.full_name} | @{user.username or 'нет'} | ID: {user.id}\n\n
            f"💰 {d.get('rub_price')} ₽ ({d.get('yuan_price'):.0f} ¥)\n"
            f"📏 {d.get('size')} | 🎨 {d.get('color')} | 🔢 {d.get('quantity')}\n
            f"📞 {d.get('contacts')}\n\n"
            f"Статус: `/status {order_id} paid` — после оплаты"
        ),
        parse_mode="Markdown"
    )

    # Шаг 7 — реквизиты
    keyboard = [[InlineKeyboardButton("🏠 Главное меню", callback_data="back")]]
    await query.edit_message_text(
        f"✅ Заказ *{order_id}* оформлен!\n\n"
        f"{PAYMENT_DETAILS}\n\n"
        "Статус отслеживай в разделе *«Мои заказы»* 📋",
        reply_markup=InlineKeyboardMarkup(keyboard),
        parse_mode="Markdown"
    )
    context.user_data.clear()
```

```
    return MAIN_MENU

# ============================================================
# МОИ ЗАКАЗЫ
# ============================================================

async def my_orders(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    orders = get_user_orders(query.from_user.id)

    if not orders:
        keyboard = [
            [InlineKeyboardButton("📸 Заказать из Китая", callback_data="order_ph
            [InlineKeyboardButton("⬅️ Назад", callback_data="back")],
        ]
        await query.edit_message_text(
            "📋 У тебя пока нет заказов.\n\nОформи первый!",
            reply_markup=InlineKeyboardMarkup(keyboard)
        )
        return MAIN_MENU

    text = "📋 *Твои заказы:*\n\n"
    for oid, o in sorted(orders.items(), reverse=True):
        status = STATUS_LABELS.get(o.get("status", "new"), "🆕 Новый")
        text += f"*{oid}*\n"
        text += f"💰 {o.get('rub_price', '?')} ₽ | {status}\n"
        text += f"📅 {o.get('date', '?')}\n\n"

    keyboard = [[InlineKeyboardButton("⬅️ Назад", callback_data="back")]]
    await query.edit_message_text(text, reply_markup=InlineKeyboardMarkup(keyboar
    return MAIN_MENU

# ============================================================
# РАСЧЁТ ДОСТАВКИ
# ============================================================

async def delivery_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    keyboard = [
        [InlineKeyboardButton("👕 Футболка / Рубашка", callback_data="del_shirt")
        [InlineKeyboardButton("👖 Джинсы / Брюки", callback_data="del_pants")],
        [InlineKeyboardButton("🧥 Куртка / Пальто", callback_data="del_jacket")],
        [InlineKeyboardButton("👟 Кроссовки / Обувь", callback_data="del_shoes")]
        [InlineKeyboardButton("🎒 Сумка / Рюкзак", callback_data="del_bag")],
        [InlineKeyboardButton("📦 Несколько позиций", callback_data="del_multi")]
```

```
        [InlineKeyboardButton("⬅️ Назад", callback_data="back")],
    ]
    await query.edit_message_text(
        "🚚 *Расчёт стоимости доставки*\n\nВыбери категорию товара:",
        reply_markup=InlineKeyboardMarkup(keyboard),
        parse_mode="Markdown"
    )
    return DELIVERY_CALC

DELIVERY_PRICES = {
    "del_shirt":  ("👕 Футболка / Рубашка", "250 ₽"),
    "del_pants":  ("👖 Джинсы / Брюки",     "500 ₽"),
    "del_jacket": ("🧥 Куртка / Пальто",    "750–1 250 ₽"),
    "del_shoes":  ("👟 Кроссовки / Обувь",  "1 000–1 500 ₽"),
    "del_bag":    ("🎒 Сумка / Рюкзак",     "300–900 ₽"),
    "del_multi":  ("📦 Несколько позиций",   "менеджер рассчитает точно"),
}

async def delivery_result(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    name, price = DELIVERY_PRICES[query.data]
    keyboard = [
        [InlineKeyboardButton("📸 Заказать из Китая", callback_data="order_photo"
        [InlineKeyboardButton("⬅️ Назад", callback_data="delivery")],
    ]
    await query.edit_message_text(
        f"🚚 *Расчёт доставки*\n\n"
        f"Категория: {name}\n"
        f"💰 Стоимость доставки: *{price}*\n\n"
        "⚠️ Оплата за доставку проводится через менеджера @projectmanag3r",
        reply_markup=InlineKeyboardMarkup(keyboard),
        parse_mode="Markdown"
    )
    return MAIN_MENU

# ============================================================
# FAQ
# ============================================================

async def faq(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    keyboard = [
        [InlineKeyboardButton("📞 Написать менеджеру", callback_data="manager")],
        [InlineKeyboardButton("⬅️ Назад", callback_data="back")],
    ]
```

```
    await query.edit_message_text(
        "❓ *Частые вопросы:*\n\n"
        "⏳ *Сроки доставки:*\n"
        "Быстрая — 12-15 дней\n"
        "Обычная — 20-30 дней\n\n"
        "↩️ *Возврат:*\n"
        "Вы выбираете размер по размерной сетке. Возврат возможен только если при
        "🚚 *Оплата за доставку:*\n"
        "Проводится через менеджера @projectmanag3r",
        reply_markup=InlineKeyboardMarkup(keyboard),
        parse_mode="Markdown"
    )
    return MAIN_MENU

# ============================================================
# МЕНЕДЖЕР
# ============================================================

async def manager(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    keyboard = [[InlineKeyboardButton("⬅️ Назад", callback_data="back")]]
    await query.edit_message_text(
        "📞 Наш менеджер: @projectmanag3r\n\nНапиши напрямую — ответим быстро!",
        reply_markup=InlineKeyboardMarkup(keyboard)
    )
    return MAIN_MENU

# ============================================================
# ADMIN КОМАНДЫ
# ============================================================

async def admin_status(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if update.effective_user.id != ADMIN_ID:
        return
    args = context.args
    if len(args) < 2:
        await update.message.reply_text(
            "Использование: /status ORD-XXXX <статус>\n\n"
            "Статусы: new | confirmed | paid | in_china | shipping | arrived | de
        )
        return
    order_id = args[0].upper()
    new_status = args[1].lower()
    if new_status not in STATUS_LABELS:
        await update.message.reply_text(f"❌ Неизвестный статус. Доступные: {', '
        return
```

```
    order = get_order(order_id)
    if not order:
        await update.message.reply_text(f"❌ Заказ {order_id} не найден.")
        return
    update_order_status(order_id, new_status)
    label = STATUS_LABELS[new_status]
    client_id = order.get("user_id")
    if client_id:
        try:
            await context.bot.send_message(
                chat_id=client_id,
                text=(
                    f"📦 *Обновление по заказу {order_id}*\n\n"
                    f"Новый статус: {label}\n\n"
                    "Вопросы? Пиши @projectmanag3r"
                ),
                parse_mode="Markdown"
            )
        except Exception as e:
            logger.error(f"Не удалось уведомить клиента: {e}")
    await update.message.reply_text(f"✅ Статус {order_id} → {label}")

async def admin_orders(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if update.effective_user.id != ADMIN_ID:
        return
    orders = load_orders()
    if not orders:
        await update.message.reply_text("Заказов пока нет.")
        return
    text = f"📋 *Все заказы ({len(orders)}):*\n\n"
    for oid, o in sorted(orders.items(), reverse=True):
        status = STATUS_LABELS.get(o.get("status", "new"), "?")
        text += f"*{oid}* | {status}\n"
        text += f"@{o.get('username','нет')} | {o.get('rub_price','?')} ₽ | {o.ge
    if len(text) > 4000:
        text = text[:4000] + "\n..."
    await update.message.reply_text(text, parse_mode="Markdown")

# ============================================================
# ОТМЕНА
# ============================================================

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data.clear()
    return await start(update, context)

# ============================================================
```

```
# ЗАПУСК
# ============================================================

def main():
    app = Application.builder().token(BOT_TOKEN).build()

    conv_handler = ConversationHandler(
        entry_points=[CommandHandler("start", start)],
        states={
            MAIN_MENU: [
                CallbackQueryHandler(catalog, pattern="^catalog$"),
                CallbackQueryHandler(order_photo_start, pattern="^order_photo$"),
                CallbackQueryHandler(delivery_start, pattern="^delivery$"),
                CallbackQueryHandler(my_orders, pattern="^my_orders$"),
                CallbackQueryHandler(faq, pattern="^faq$"),
                CallbackQueryHandler(manager, pattern="^manager$"),
                CallbackQueryHandler(cancel, pattern="^back$"),
            ],
            ORDER_PHOTO: [
                MessageHandler(filters.PHOTO | filters.Document.IMAGE, order_phot
                CallbackQueryHandler(cancel, pattern="^back$"),
            ],
            ORDER_PHOTO_WAIT_PRICE: [
                CallbackQueryHandler(photo_price_ok, pattern="^photo_price_ok$"),
                CallbackQueryHandler(cancel, pattern="^back$"),
                MessageHandler(filters.TEXT & ~filters.COMMAND, order_photo_waiti
            ],
            ORDER_PHOTO_SIZE: [
                MessageHandler(filters.TEXT & ~filters.COMMAND, order_photo_size)
                CallbackQueryHandler(size_unknown, pattern="^size_unknown$"),
            ],
            ORDER_PHOTO_SIZE_WAIT: [
                CallbackQueryHandler(size_known, pattern="^size_known$"),
                MessageHandler(filters.TEXT & ~filters.COMMAND, order_photo_size_
            ],
            ORDER_PHOTO_COLOR:    [MessageHandler(filters.TEXT & ~filters.COMMAND
            ORDER_PHOTO_QUANTITY: [MessageHandler(filters.TEXT & ~filters.COMMAND
            ORDER_PHOTO_CONTACTS: [MessageHandler(filters.TEXT & ~filters.COMMAND
            ORDER_PHOTO_CONFIRM: [
                CallbackQueryHandler(order_photo_confirm, pattern="^photo_confirm
                CallbackQueryHandler(cancel, pattern="^back$"),
            ],
            DELIVERY_CALC: [
                CallbackQueryHandler(delivery_result, pattern="^del_"),
                CallbackQueryHandler(cancel, pattern="^back$"),
            ],
        },
```

```
        fallbacks=[CommandHandler("start", start)],
    )

    app.add_handler(conv_handler)
    app.add_handler(CommandHandler("status", admin_status))
    app.add_handler(CommandHandler("orders", admin_orders))
    app.add_handler(CommandHandler("price", admin_send_price))
    app.add_handler(CommandHandler("sizechart", admin_sizechart_cmd))

    # Менеджер отправляет размерную сетку — ловим фото/документ вне диалога
    app.add_handler(MessageHandler(
        filters.User(ADMIN_ID) & (filters.PHOTO | filters.Document.ALL),
        admin_sizechart_send
    ))

    logger.info("Бот запущен...")
    app.run_polling(drop_pending_updates=True)

if __name__ == "__main__":
    main()
```